

Lab 3: Architecture Justification

Problem Statement #43: Feature Flag & Dynamic Config Manager

Architecture Selection

We chose **Client-Server Architecture** for the **Feature Flag & Dynamic Config Manager**.

1. Architectural Choice

A centralized server tier (API Gateway & Access Control, Cohort Manager, Rollout Manager, Environment Config Manager, Flag Evaluation Engine, SDK Sync Service, Flag & Config Store) serves two kinds of clients: the Admin Console used by the Software Engineer and Release Manager, and the Client SDKs embedded in applications. The server is modular internally, but it is deployed as one tier. Layered and Microservices were evaluated and rejected: Layered gives clean separation but tightly couples layers and is hard to scale, while Microservices gives independent scaling but adds network hops and operational complexity.

2. Reason 1: Rollback and sync deadlines (NFR-001, NFR-002, FR-005, FR-002)

NFR-002 requires flag updates to reach client SDKs in under 12 ms, and NFR-001 requires a faulty feature to be disabled on all connected devices within 120 seconds. Both are push problems with one source and many receivers. In Client-Server, a rollback written by the Rollout Manager is picked up by the SDK Sync Service through the Store's *IFlagChangeFeed* and pushed straight to every connected SDK over *ISDKStream*. A Microservices design would add inter-service network calls to this path, which is exactly the latency we cannot afford against a 12 ms budget.

3. Reason 2: One authoritative flag state (FR-001, FR-003, FR-004)

FR-003 requires a feature to be ON in Testing but OFF in Production, and FR-004 requires one rollout percentage per feature applied to a cohort (FR-001). These only work if every decision reads the same data. A single Flag & Config Store behind one server guarantees that the Admin Console and all SDKs see consistent cohorts, environment states and percentages, without the cross-service data consistency problems of Microservices.

4. Security Advantage

Every administrative request enters through one choke point, the API Gateway & Access Control, which enforces the Lab 1 precondition that only users with permission may change rollout settings. The SDK Sync Service validates each SDK connection through *IAccessControl*. Clients never reach the Flag & Config Store directly, so cohort lists and targeting rules stay on the server.

5. Performance Benefit

Server components are co-deployed, so calls between the Rollout Manager, Flag Evaluation Engine and Store are local calls rather than network calls between separate services. Together with persistent push connections (no SDK polling), the path from a flag change to the SDK stays short, which is what makes the 12 ms target in NFR-002 achievable. The cost is a single point of failure and a scaling bottleneck, which we accept and would address by running replicas of the server tier.